

Distributed Systems: Consensus and Raft

CISS 2026WS - Exam-focused course notes

1. Consensus goals and terminology

Consensus allows a group of processes to agree on one value despite failures. A correct consensus protocol provides agreement (no two correct processes decide differently), validity (the decided value was proposed), and termination (every correct process eventually decides).

Safety properties state that nothing bad happens; they must hold even during network delays.

Liveness properties state that something good eventually happens; they generally require timing or availability assumptions.

Crash faults stop a process but do not make it lie. Byzantine faults permit arbitrary behavior and require stronger protocols than Raft.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

2. Raft roles, terms, and communication

Raft servers are followers, candidates, or leaders. Normal operation has one leader; clients send commands to that leader. Followers are passive and respond to requests.

Time is divided into monotonically increasing terms. Each term begins with an election. A term has at most one leader, but it may end without electing one.

RequestVote RPCs are used for elections. AppendEntries RPCs replicate log entries and also serve as heartbeats.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

3. Leader election

A follower becomes a candidate after its randomized election timeout expires without a valid leader message. It increments its term, votes for itself, resets its timer, and requests votes from the other servers.

A candidate becomes leader after receiving votes from a majority. Each server grants at most one vote per term. Randomized election timeouts reduce repeated split votes.

A voter grants its vote only when the candidate's log is at least as up to date as its own. Raft compares last-log term first, then last-log index.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

4. Log replication and commitment

The leader appends a client command to its log, sends `AppendEntries` to followers, and retries until followers contain matching entries. An entry is committed after it is stored on a majority and the leader's commitment rule is satisfied.

Raft leaders commit entries from their current term by counting replicas. Once such an entry is committed, preceding entries are committed indirectly, including entries from older terms.

Servers apply committed entries to the state machine in log order. The commit index is monotonic and never decreases.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

5. Log matching and conflict repair

The Log Matching Property says that if two logs contain an entry with the same index and term, the entries are identical and all preceding entries are identical.

AppendEntries includes the previous log index and term. A follower rejects the request when that previous entry does not match. The leader then backs up its nextIndex and retries.

When a matching prefix is found, conflicting follower entries are deleted and replaced by the leader's entries. A leader never overwrites or deletes its own entries.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

6. Raft safety properties

Election Safety: at most one leader can be elected in a term. Leader Append-Only: a leader never changes existing entries in its own log. Log Matching: equal index and term imply equal prefixes.

Leader Completeness: an entry committed in a term appears in the logs of leaders for all higher terms. State Machine Safety: no two servers apply different commands at the same log index.

The voting up-to-date restriction is essential for Leader Completeness because a candidate missing a committed entry cannot collect a majority that includes enough replicas of it.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

7. Membership changes and snapshots

Changing directly from one server set to another can create two independent majorities. Raft avoids this using joint consensus, where decisions require majorities of both the old and new configurations during the transition.

Snapshots compact the prefix of the replicated log after its effects are represented in the state machine snapshot. The snapshot records the last included index and term.

A leader sends InstallSnapshot when a follower is too far behind for retained log entries to repair it efficiently.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

8. Worked election and replication example

In a five-server cluster, three votes form a majority. If server A receives votes from A, B, and C in term 9, it becomes leader even if D and E are unreachable.

For a term-9 command to be committed by counting replicas, the entry must be stored on at least three servers. Losing two servers still leaves a majority available; losing three prevents progress but does not permit conflicting committed values.

Exam distinction: majority replication determines commitment, while application occurs after commitment and in log order. Replicated, committed, and applied are different states.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.